

- Программирование, как процесс процедурного решения задач
- Понятие Алгоритм
- Процесс проектирования и анализа алгоритмов
- Понятие Язык программирования
- Понятие Среда разработки, интегрированная среда разработки, vs код
- Структуры программы на языке программирования на примере языка C++

Программирование, как процесс процедурного решения задач

Считаю, что корректнее для начала разобраться, что такое программирование в целом

Программирование - это процесс создания компьютерных программ для решения каких-либо задач

Мы прекрасно понимаем, что задачи у нас могут быть разнообразные, начиная от ускорения какого-то процесса заканчивая созданием какого-то продукта (программы), который закрывает новые потребности человечества. Употребляла термин "программа", поэтому даю определение

Программа — это набор инструкций, предписаний или планов, определяющих последовательность действий для достижения определённой цели

Программисты - люди умные и давным-давно поняли, что для разных поставленных задач, нужно использовать разные подходы (очевидно?). Их давно придумали, разработали и классифицировали. А обозвали эти подходы красивым названием "Парадигмы"

Парадигмы - набор идей, концепций и правил, определяющих стиль написания компьютерных программ и способ организации вычислений

У каждой парадигмы есть свои правила, стандарты и понятия, которым обязан следовать разработчик. Хочу также отметить, что одна и та же задача может быть решена разными парадигмами, так что периодически

перед вами будет стоять задача "С помощью какой парадигмы мне решить эту задачу?"

Классификация парадигм

Сейчас будет длинный текст, разъясняющий что для чего, потом увидите красивую картинку, благодаря которой вам станет все понятно

Две основные парадигмы: императивная и декларативная, которые вмещают в себя остальные (то есть остальные парадигмы - подтипы этих двух)

= Императивная

Самая простая и часто используемая парадигма. Её смысл в последовательном выполнении действий. Её основные подтипы, которые для вас будут иметь интерес: Процедурное и Объектно-ориентированное программирование (ООП)

- Процедурное

Основным понятием в процедурном программировании является функция (процедура)

Функция (или процедура) - последовательность действий, которую записали и назвали

В этой парадигме вы будете работать в этом семестре

- ООП

В этой парадигме есть такое понятие, как объект, который обладает сам какими-то свойствами и функциями. Об этом подробнее в следующем семестре.

P.S. Пример процедурного: вам прислали шкаф с инструкцией по сборке. В данной ситуации, инструкция к шкафу - есть функция (процедура)

Если говорить про ооп, то в этом подходе у нас есть объект шкаф, который можно собрать, разобрать и т.д. В этой парадигме центром программы будут объекты и их взаимодействие, а не сами функции

= Декларативная

В этой парадигме разработчику сосредотачивается на результате, а не на самих действиях. Т.е. разработчик дает команду, а компьютер ищет сам решение для этой задачи. Если в императивной разработчик обязательно самостоятельно указывает последовательность действий, то в декларативной система сама решит как это выполнить.

- Функциональное программирование

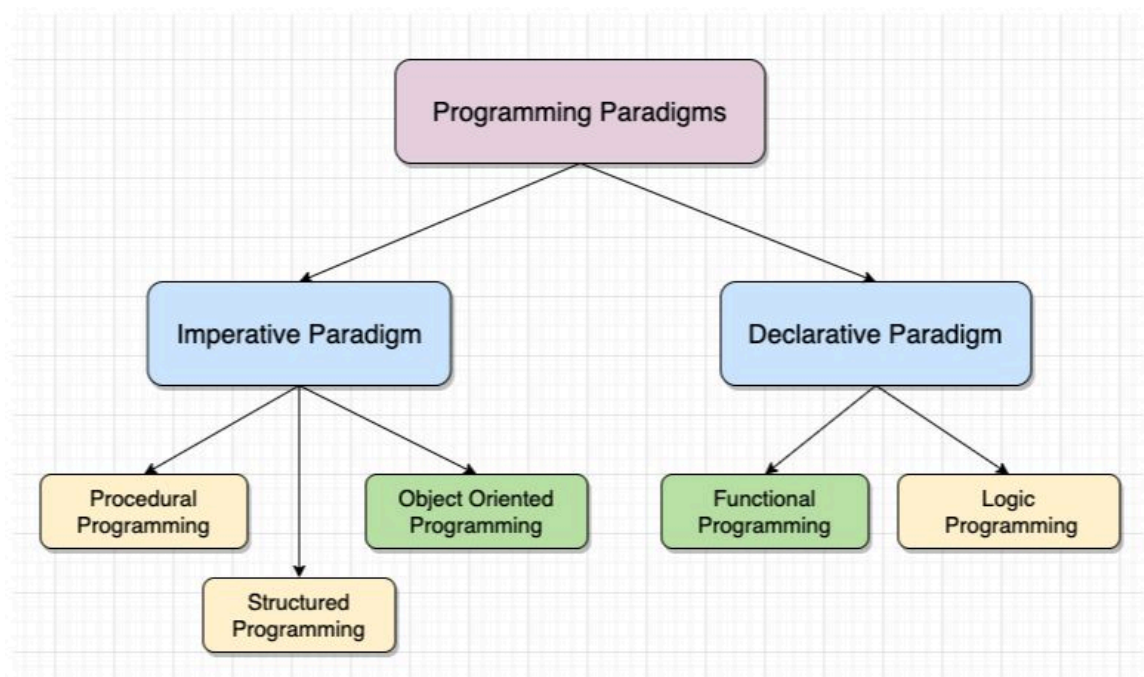
НЕ ПУТАТЬ С ПРОЦЕДУРНЫМ, ЭТО РАЗНЫЕ ПАРАДИГМЫ. В строгом функциональном подходе считается, что все функции чистые и не имеют побочных эффектов. Функция никогда не меняет объект, а возвращает

всегда новый

- Логическое программирование

Это такой подход к созданию программ, основанный на принципах математической логики, где программа представляет собой набор фактов и правил вывода, а ее выполнение заключается в автоматическом выводе заключений на основе этих фактов

В заключении по этому блоку хочу отметить, что есть концепции и принципы, которые пересекаются в языках. Для более подробного ознакомления рекомендую погуглить принципы *такой-то* парадигмы



Таким образом, делаем вывод по этому блоку: В этом семестре вам предстоит писать программы в концепции процедурного программирования

Понятие алгоритм + Процесс проектирования и анализа алгоритмов

Приступаем ко второму блоку этой лекции. Обсудим само понятие, зачем они нужны, виды алгоритмов, про блок-схемы и сложность и еще куча слов, которые могут быть не очень понятны. Давайте по порядку

Алгоритм — это четкая последовательность действий, выполнение которой дает какой-то заранее известный результат

Т.е. мы имеем какой-то набор инструкций для решения задачи. Назревает вопрос зачем? Если не вдаваться в философию, что любая инструкция в нашей жизни это алгоритм, то можно вспомнить задачи, которые в лоб либо вообще не решаются, либо решаются очень неэффективно. Такие сложные задачи уже решили за нас математики, наверняка вы слышали про такие громкие алгоритмы по типу алгоритма Дейкстры, Евклида, Ньютона и подобные. Алгоритмы достаточно большой раздел математики, в первую очередь, поэтому рекомендуется ознакомиться с ним самостоятельно, особенно если вы метите в крупные компании.

Виды алгоритмов:

- Линейные

Действия идут друг за другом, выполняются при любых условиях

- Ветвящиеся

Действия выполняются в зависимости от условия. Наш любимый if-else

- Циклические

Какой-то блок действий повторяется некоторое количество раз

- Рекурсивные

Алгоритм вызывает сам себя, но с другими входными данными. Вы можете подумать, что цикл уж очень похож на рекурсию и будете правы, однако, цикл - частный случай рекурсии, поэтому приравнивать их некорректно

- Вероятностные

Алгоритм учитывает случайные величины

Важно отметить, что это не все типы и периодически алгоритм достаточно сложно классифицировать, поэтому чаще их разделяют по какому-то типу задач или типу данных. Например, алгоритм DFS (deep first search) алгоритм обхода графа от начальной вершины до конечной. Короче, деление очень условное, поэтому запоминать их виды не является острой необходимостью.

А вот определять сложность алгоритма - очень полезный и важный навык. Под анализом алгоритмов как раз и подразумевают оценку временной и пространственной сложности

Сложность алгоритма - количественная мера ресурсов, необходимых алгоритму для решения задачи, в зависимости от размера входных данных

Под ресурсами мы имеем ввиду время, за которое алгоритм завершит работу и затраты памяти. Основная задача программиста при разработке

алгоритма (или выбора алгоритма) - использовать алгоритм максимально эффективный (логично). В общем, оценкой алгоритма занимается математический метод "Асимптотический анализ". Он включает в себя несколько подвидов анализа (Big O, Big Theta, Big Omega и т.д.), подробнее про них вы найдете на просторах интернета. Программисты решили сосредоточиться на Big O Notation, который оценивает самый худший результат или же верхнюю границу значений ресурсов. Именно поэтому на каком-нибудь литкоде вы увидите оценку сложности $O(n)$.

Какие бывают сложности:

- $O(1)$ означает, что алгоритм выполняется за фиксированное константное время. Это самые эффективные алгоритмы.
- $O(n)$ — это сложность линейных алгоритмов. n здесь и дальше обозначает размер входных данных: чем больше n , тем дольше выполняется алгоритм.
- $O(n^2)$ тоже означает, что чем больше n , тем выше сложность. Но зависимость тут не линейная, а квадратичная, то есть скорость возрастает намного быстрее. Это неэффективные алгоритмы, например с вложенными циклами.
- $O(\log n)$ — более эффективный алгоритм. Скорость его выполнения рассчитывается логарифмически, то есть зависит от логарифма n .
- $O(\sqrt{n})$ — алгоритм, скорость которого зависит от квадратного корня из n . Он менее эффективен, чем логарифмический, но эффективнее линейного.

Существуют также $O(n^3)$, $O(nn)$ и другие малоэффективные алгоритмы с высокими степенями. Их сложность растет очень быстро, и их лучше не использовать.

Если вы сейчас задумались, как по этим буквам понять что эффективнее, зайдите в десмос и посмотрите, какой из графиков растет быстрее. Ну или возьмите производную, если знаете как))

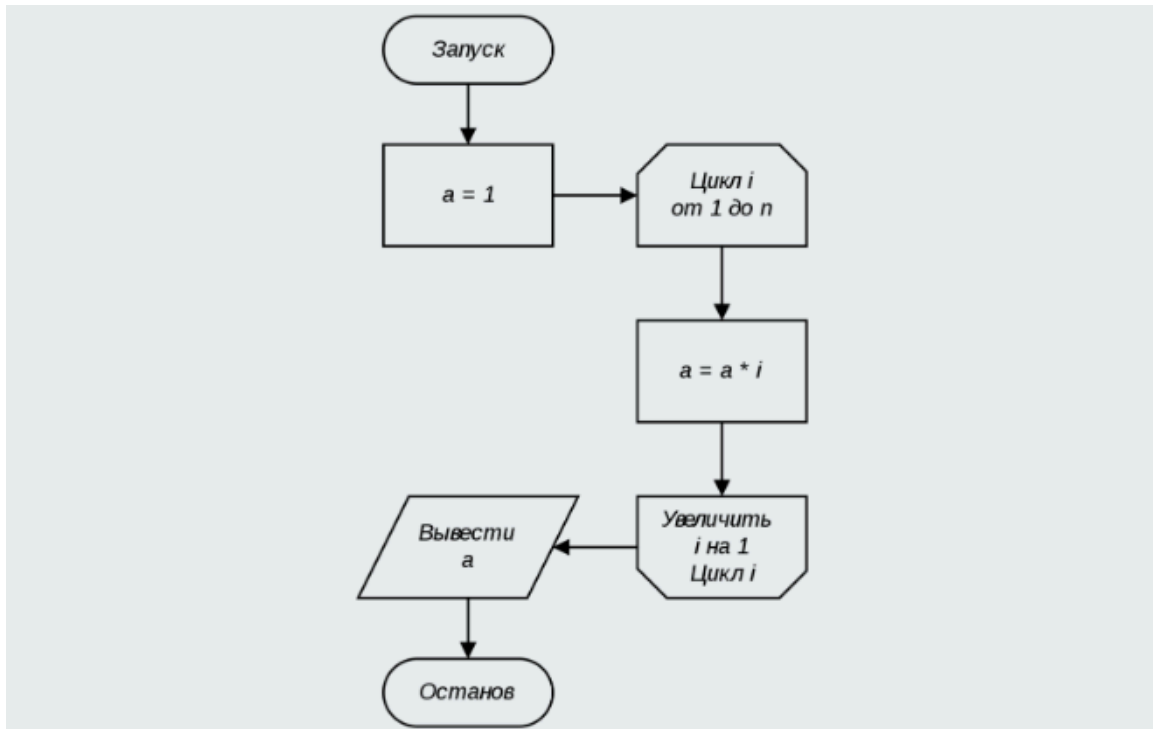
!В программировании, информатики и т.п. под $\log$ подразумевается основание 2

Шаги разработки алгоритма:

1. Идентификация проблемы или задачи
2. Разделение задачи на подзадачи
3. Определение последовательности действий
4. Применение логических конструкций
5. Объединение подзадач в общий алгоритм

Логично, что после разработки алгоритма, мы его напишем на каком-нибудь Python или на другом языке программирования (о них чуть позже). Но что если человек не знает этого языка, но очень хочет узнать, что же вы такое придумали? Для решения этой проблемы есть текстовое описание алгоритма, шедевро псевдо код и самое прикольное - блок-схема. На ней остановимся поподробнее.

Блок-схема - специальная схема, состоящая из геометрических фигур, которые описывают те или иные действия алгоритма



На этой картинке пример блок-схемы и, как вы могли заметить, фигуры тут разные. Например, ромб - данные, прямоугольник - простое действие и т.д. Стрелки обозначают направление. Регламентируется это ГОСТ 19.701-90, ввиду того, что этих обозначений достаточно много, все я выписывать не буду, извините. (Если читаете это из репозитория, то можете рядом в этой же папке этот гост увидеть, в противном случае, загуглите)

Понятие язык программирования

Язык программирования (ЯП) — это формальная знаковая система, на которой пишут компьютерные программы

Для компьютера существует один единственный понятный ему код - двоичный. Но писать на нем не очень удобно, не находите? Для этого и были созданы языки программирования. Классифицировать ЯП можно двумя способами:

Компилируемые и Интерпретируемые ЯП

Они различаются принципом работы. Компилятор получает на вход код и компилирует его — собирает, преобразует в исполняемый файл весь разом. Потом этот исполняемый файл можно запустить, и компьютер поймет, что от него хотят. Интерпретатор «переводит» код построчно и тут же иницирует его выполнение. Сначала он прочитает и отправит на исполнение первую строчку, затем вторую - и так далее. Такая классификация важна для отладки кода.

Немного подробнее про этапы компиляции, т.к. процесс это сложный, но безумно важный для понимания

1. *Препроцессирование.* Вставление директив, статических библиотек и модулей
2. *Трансляция.* «Трансляция» — это (с английского) «перевод». Кого и куда переводим? Наш язык в ассемблер. (это язык, самый близкий к машинному коду)
3. *Ассемблирование.* На данном этапе не происходит ничего интересного — просто инструкции, которые были в ассемблере, перегоняются в машинный код.
4. *Линковка.* На выходе даёт исполняемый файл. Нужна, если файлов несколько: мы запускаем препроцессор, трансляцию и ассемблирование независимо для каждого файла, а объединяются они только на этапе линковки.
5. *Загрузка.* Последний этап, который предстоит пройти нашей программе — вызвать загрузчик для загрузки нашей программы в память. На данной стадии также возможна подгрузка динамических библиотек.

Низкоуровневые и Высокоуровневые ЯП

У низкоуровневых языков нет встроенных функций. Они общаются с конкретным процессором, поэтому код одного компьютера не может быть использован на другом. У каждого производителя процессоров свой собственный набор инструкций. Самые распространенные низкоуровневые языки — это машинный код и язык ассемблера.

Язык ассемблера упрощает машинный код, но он все еще сложен для

массового использования. Поэтому программисты начали создавать новые языки, которые больше упрощают программирование. Так появились высокоуровневые языки, синтаксис которых близок к английскому. Важно понимать нюанс- чем ниже язык по уровню, тем больше свободы и меньше затрат по времени и памяти языку необходимо. Например, одна и та же программа на ассемблере будет весить 16 кб, на C++ 4 мб, а на питоне уже 16 мб. Складывается впечатление, что питон нужно удалить из жизни всех людей по такой логике. Однако я не зря сказала про свободу. На ассемблере вы можете удалить папку с ОС, а на питоне это невозможно. На плюсах у вас больше возможностей по управлению памятью, а на том же питоне у вас чисто физически это не получится. Сильное преимущество таких языков как C++ - возможность выделять память, управлять ею, а также удалять. Для некоторых задач это крайне важно, поэтому мы вас и знакомим с таким чудесным языком с большим количеством возможностей

!На самом деле границу по высокому\низкому уровню установить практически невозможно, банально в виду того, что только машинный код компьютер понимает, поэтому много значения этой классификации не придавайте.

Понятие Среда разработки, Интегрированная среда разработки, VS Code

С ЯП разобрались, но их же где-то надо писать? Для этого придумали IDE

IDE (от англ. Integrated Development Environment, «интегрированная среда разработки») — это программа, в которой разработчики пишут, проверяют, тестируют и запускают код, а также ведут большие проекты.

Она включает в себя сразу несколько инструментов: редактор для написания кода, сервисы для его проверки и запуска, расширения для решения дополнительных задач разработки. Можно сказать, что это как Photoshop для дизайнера — общее пространство для большинства рабочих процессов. Возможно, вы слышали про такую штуку как редактор кода или текстовый редактор, однако это не одно и то же. Редактор кода — это простой инструмент. Его задача — обеспечить программисту удобное пространство для написания кода, с подсветкой синтаксиса и автозаполнением отдельных функций. В них, как правило, довольно мало дополнительных инструментов, и часто их требуется подключать отдельно. Обычно запускать код в них можно только с помощью

расширений.

В данном курсе вам предложено ознакомиться с VS Code

!Visual Studio Code (VS Code) — это редактор исходного кода. Его разработал Microsoft для всех популярных операционных систем: Windows, Linux и macOS.

Только что засрала редактор кода и предлагаю вам вс код. Но на самом деле он представляет собой мощный редактор кода с функциями IDE, который можно расширить до полноценной среды разработки с помощью плагинов.

Структуры программы на языке программирования на примере языка C++

Уже не терпится увидеть C++, но акцентирую внимание на следующем моменте: общей структуры для всех ЯП не существует, да, есть какие-то общие правила и тенденции, но вы их заметите при знакомстве с другими языками, поэтому говорить за все ЯП не буду

```
In [ ]: #include <iostream>                // подключаем заголовочный файл iostream

int main()                                // определяем функцию main
{                                          // начало функции
    std::cout << "Hello World!";        // выводим строку на консоль
    return 0;                            // выходим из функции
}                                          // конец функции
```

Это тот самый "Hello World!", который пишут все в самом начале. На нем разберемся с структурой программы на C++

Инструкции

Программа на C++ состоит из набора инструкций. Каждая инструкция (statement) выполняет определенное действие. В конце инструкции в языке C++ ставится точка с запятой (;). Данный знак указывает компилятору на завершение инструкции. Например:

```
In [ ]: std::cout << "Hello World!"; //является инструкцией и поэтому завершается тс

In [ ]: {
    std::cout << "Hello World!"; //чтобы определить блок кода мы его заключа
    std::cout << "Bye World!";   //а инструкции располагаем между ними
}
```

Функция main

Каждая программа на языке C++ должна иметь как минимум одну

функцию - функцию `main()`. Именно с этой функции начинается выполнение приложения. Имя `main` фиксировано и для всех программ на C++ всегда одинаково.

Функция также является блоком кода, поэтому ее тело обрамляется фигурными скобками, между которыми определяется набор инструкций. `int main()` мы как раз видим в нашем примере

Определение функции `main` начинается с возвращаемого типа. Функция `main` в любом случае должна возвращать число, поэтому тип `int`. Далее идет название функции, то есть `main`. После названия в скобках идет список параметров. В данном случае функция `main` не принимает никаких параметров, поэтому после названия указаны пустые скобки.

Далее идут как раз наши действия

В конце функции идет инструкция `return`, обозначающая завершение выполнения функции и возвращающая какое-то значение в месте, где была вызвана

`main` запускает всегда, это, буквально, тело нашей программы, которая возвращает значение, внимание, самой ОС

Число 0 после оператора `return` указывает операционной системе, что выполнение функции завершилось успешно, без ошибок.

Директивы препроцессора

В примере выше на консоль выводится строка, но чтобы использовать вывод на консоль, необходимо в начале файла с исходным кодом подключать библиотеку `iostream` с помощью директивы `include`.

В отличие от обычных инструкций языка C++, которые завершаются точкой с запятой `;`, признаком завершения препроцессорной директивы является перевод на новую строку. Кроме того, директива должна начинаться со знака решетки `#`. Непосредственно директива `"include"` определяет, какие файлы и библиотеки надо подключить в данном месте в код программы.

Комментарии

Комментарии нужны как чужим людям, смотрящим ваш код, так и вам самому, чтобы понять, что вы написали в 5 утра

Комментарии игнорируются компилятором и не влияют на решение, вес и время выполнения кода.

В языке C++ есть два типа комментариев: однострочный и многострочный. Однострочный комментарий размещается на одной строке после двойного следа `//` (его вы уже видели)

Многострочный комментарий заключается между символами `/*` текст комментария `*/`. Он может размещаться на нескольких строках.

```
In [ ]: #include <iostream>
        /*
           Определение функции Main
           Выводит на консоль строку Hello World!
        */
        int main()
        {
            std::cout << "Hello World!"; // вывод строки на консоль
            return 0;
        }
```

Всем спасибо за внимание

This notebook was converted with convert.ploomber.io